```
"scripts": {
    "start:provider": "cross-env PORT=3001
node provider/provider-service.js",
    "start:graphql:server": "npm run start:provider &
node graphql/provider/graphql-server.js"
  }
```

The command also starts the movie API provider because our GraphQL server has a dependency on it.

7.2.3 Consumer overview

From the GraphQL consumer perspective, we're also keeping it simple. A user interface (UI), typically using React, which is a popular JavaScript framework for building web applications, presents a list of movies in real-world projects. In our simple GraphQL project, we expose the functions the UI needs to display the data.

Inside the graphql folder, you should see another folder named consumer, which contains a file called graphql-client.js. Let's break down the code into two parts, with the first part focused on setting up the `ApolloClient`:

```
const { ApolloClient, InMemoryCache, gql, HttpLink }
= require('@apollo/client');

const client = new ApolloClient({             #1
  cache: new InMemoryCache({                  #1
    addTypename: false,                       #1
  }),                                         #1
  link: new HttpLink({                        #1
    fetch: require('node-fetch'),             #1
    uri: 'http://127.0.0.1:4000/graphql',     #1
  }),                                         #1
});                                           #1
```

#1 Initializes ApolloClient so we can fetch data from our GraphQL server

The next part of the code defines the queries that we need to fetch all the movies and a single movie from our GraphQL server: